```
const os = require('os')


// os.uptime()
const systemUptime = os.uptime();
// os.userInfo()
const userInfo = os.userInfo();


// We will store some other information about my WindowsOS in this object:
const otherInfo = {
    name: os.type(),
    release: os.release(),
    totalMem: os.totalmem(),
    freeMem: os.freemem(),
}


// Let's Check The Results:
console.log(systemUptime);
console.log(userInfo);
console.log(otherInfo);
```

—---------

```
var http = require('http');

http.createServer(function (req, res) {
  res.writeHead(200, {'Content-Type': 'text/html'});
  res.end('Hello World!');
}).listen(8080);
```

—----------

myfirstmodule.js

```
exports.myDateTime = function () {

  return Date();

};
```

App.js

```
var http = require('http');

var dt = require('./myfirstmodule');

http.createServer(function (req, res) {

  res.writeHead(200, {'Content-Type': 'text/html'});

  res.write("The date and time are currently: " + dt.myDateTime());

  res.end();

}).listen(8080);
```

Split the query string into readable parts:

first.js

```
var http = require('http');

var url = require('url');

http.createServer(function (req, res) {

  res.writeHead(200, {'Content-Type': 'text/html'});

  var q = url.parse(req.url, true).query;

  var txt = q.year + " " + q.month;

  res.end(txt);

}).listen(8080);
```

```
> node first.js
```

The address:

http://localhost:8080/?year=2017&month=July

//json format

```
var http = require('http');

http.createServer(function (req, res) {
  res.writeHead(200, { 'Content-Type': 'application/json' });

  const data = {
```

```
    message: "Hello World!",
    status: "success",
    time: new Date().toISOString()
  };

  res.end(JSON.stringify(data));
}).listen(8080);

console.log("Server running at http://localhost:8080/");
```

—----------

```
//image format
var http = require('http');



var fs = require('fs');
var path = require('path');

http.createServer(function (req, res) {
  // Example: store  1.jpg in the same folder
  const imagePath = path.join(__dirname, '1.jpg');

  fs.readFile(imagePath, function (err, data) {
    if (err) {
      res.wr      res.writeHead(200, { 'Content-Type'iteHead(404, { 'Content-Type': 'text/plain' });
      res.end('Image not found');
    } else {
: 'image/jpeg' });
      res.end(data); // send image binary data
    }
  });
}).listen(8080);

console.log("Server running at http://localhost:8080/");
```

—----------

```
const EventEmitter = require('events');

// Create an instance of EventEmitter
const myEmitter = new EventEmitter();

// Add an event listener to the 'greet' event
myEmitter.on('greet', () => {
  console.log('Hello, Node.js!');
});

// Emit the 'greet' event
myEmitter.emit('greet');
```

—---------

```
const EventEmitter = require('events');
const myEmitter = new EventEmitter();

// Add a listener for the 'message' event
function onMessage(msg) {
  console.log(`Received: ${msg}`);
}

myEmitter.on('message', onMessage);

// Remove the listener
myEmitter.removeListener('message', onMessage);

// Emit the event (nothing happens because the listener was removed)
myEmitter.emit('message', 'This will not be received');
```

—----------

```
const fs = require('fs');
const EventEmitter = require('events');

class FileWatcher extends EventEmitter {
  constructor() {
    super();
  }

  watch(file) {
    fs.watch(file, (eventType, filename) => {
      if (eventType === 'change') {
        this.emit('fileChanged', filename);
      }
    });
  }
}

// Create an instance of FileWatcher
const fileWatcher = new FileWatcher();

// Listen for the 'fileChanged' event
fileWatcher.on('fileChanged', (filename) => {
  console.log(`File changed: ${filename}`);
});

// Start watching a file
fileWatcher.watch('test.txt');
```

Testing:
1. Create a file called `test.txt`.
2. Modify the file while the script is running, and you should see the console log `File changed: test.txt` when it detects the change.

—----------

```
const EventEmitter = require('events');
const myEmitter = new EventEmitter();
```

```
// Initialize a counter to track the number of 'click' events
let clickCount = 0;


// Add a listener for the 'click' event
myEmitter.on('click', () => {
  clickCount++;
  console.log(`Button clicked ${clickCount} times`);
});

// Simulate button clicks
myEmitter.emit('click');
myEmitter.emit('click');
myEmitter.emit('click');
```

—----------

Publisher and Subscriber in different files

publisher.js

```
const EventEmitter = require('events');
class MyEmitter extends EventEmitter {}

const emitter = new MyEmitter();

// Function to publish events
function publishMessage(channel, message) {
  console.log(`Publishing on channel: ${channel}`);
  emitter.emit(channel, message);
}

// Export both emitter and publisher function
module.exports = { emitter, publishMessage };
```

subscriber.js

```
const { emitter } = require('./publisher');
```

```
// Subscribe to "news" channel
emitter.on('news', (data) => {
  console.log(`Subscriber 1 received news: ${data}`);
});

// Subscribe to "updates" channel
emitter.on('updates', (data) => {
  console.log(`Subscriber 2 received update: ${data}`);
});
```

app.js

```
const { publishMessage } = require('./publisher');
require('./subscriber');  // load subscribers

// Publish some messages
publishMessage('news', 'Breaking: Node.js Pub/Sub implemented!');
publishMessage('updates', 'Version 1.0 released.');
```

—----------

```
//delayed message
const EventEmitter = require('events');
const myEmitter = new EventEmitter();

// Add a listener for the 'delayed' event
myEmitter.on('delayed', (message) => {
  console.log(`Received delayed message: ${message}`);
});

// Emit a 'delayed' event after 3 seconds
setTimeout(() => {
  myEmitter.emit('delayed', 'This is a delayed message!');
}, 3000);
```

—----------

```
//job queue where jobs are processed as events.


const EventEmitter = require('events');
const myEmitter = new EventEmitter();

// Define a job queue
let jobQueue = [];

// Add a job to the queue
function addJob(job) {
  jobQueue.push(job);
  myEmitter.emit('newJob', job);
}

// Listen for the 'newJob' event and process jobs
myEmitter.on('newJob', (job) => {
  console.log(`Processing job: ${job}`);
  setTimeout(() => {
    console.log(`Job ${job} completed`);
  }, 1000);
});

// Add jobs to the queue
addJob('Job 1');
addJob('Job 2');
addJob('Job 3');
c
```